5 – FUNCTIONS

Functions are a fundamental control structure that allow you to give a name (*a label*) to a reusable block of code. Once defined, that block can be reused anywhere in your program simply by calling its name. This promotes code reuse, modularity, reduces duplication, and lowers the chance of errors.

You can think of a function as a small program with a **single responsibility**. A function may:

- accept inputs (called *parameters* or *arguments*),
- perform some logic, and
- optionally return one or more output values.

Functions can call other functions, and in some cases, even call themselves (a technique known as *recursion*).

Using functions naturally encourages **problem decomposition**—breaking a complex problem into smaller, simpler tasks. Each task is easier to implement, reason about, test, and maintain.

Python supports:

- **default (optional) arguments**
- **keyword arguments**
- **arbitrary arguments** (a variable number of arguments)
- **anonymous functions** (functions without names, usually created with lambda)

Unlike most modern languages, python does not support function overloading i.e. multiple functions with the same name. If you define multiple functions with the same name, only the last definition remains active. You can partially simulate function overloading by using default arguments.

You may safely skip the section on Higher-order functions and Closures (section 1.c.4.3 and 1.c.4.4).

5.a – User-Defined Functions

5.a.1 – Creating Functions

Functions are defined using the `def` keyword, followed by:

1. the function name,
2. parentheses containing optional parameters, and

3. an indented block that forms the function body.

```
def foo():
  print('Programming 1 is the best course')
```

Indentation is mandatory and tells Python which statements belongs to the function.

5.a.2 – Calling Functions

To execute a function, write its name followed by parentheses.

```
foo()
foo()
#prints the message twice
```

5.a.3 – Functions with Inputs

Functions become far more useful when they accept inputs. Each call can then produce different behavior depending on the provided arguments.

```
def add(first, second):
  print(first + second)

add(2, 3)                #will output 5
```

5.a.4 – Functions the explicitly return a value

A function can return a value using the `return` keyword. Returned values can be captured in variables or used in expressions.

> Strange fact
>
> Return value of a function is not the same as what is printed to the screen.

```
def calculate_tax(price):
  return price * 0.15

tax = calculate_tax(2)    #tax is calculated on $2.00
```

Note: Functions without an explicit return statement automatically returns None.

```
def foo():
  a = 1 + 3

print(foo())        # outputs None
```

5.a.5 – Variable Scope

Variables defined inside a function exist only while that function is executing. Where a variable is defined determines *where it can be accessed*.

```
a = b = c = 1             #global scope
def foo():
  global a
  a, b = 2, 2             #b is local
  print(f'{a} {b} {c}')

foo(n)                    #Prints 2 2 1
print(f'{a} {b} {c}')     #Prints 2 1 1
```

Use the `global` keyword sparingly—it can make programs harder to reason about.

5.a.6 – Keyword Arguments

When calling a function, you can provide a keyword argument or simply just the value. Using a keyword argument means that you don't have to follow any order when providing the inputs.

```
def divide(dividend, divisor):
  result = dividend / divisor

divide(10, 5)                    #Option 1:
divide(divisor =5, dividend =10) #Option 2:
```

5.a.7 – Default Arguments

If the value of one of the arguments to a function is normally unchanged, then you can simplify the calling of the function by setting a default value for that particular argument. The `split()` and `print()` methods have default argument of `' '` and `'\n'` respectively.

```
def greet(name, greeting = 'Hi'):
  print(f'{greeting} {name}')

greet('Ilia')             #output: Hi Ilia
greet('Hao', 'Hello')     #output: Hello Hao
```

5.a.8 – Annotation

Python support function annotations for both parameter and return type. If provides additional information about a function's argument and return type. This metadata is specified using a colon (`:`) for parameters and an arrow (`->`) for the return type. This information is stored in the function's `__annotations__` attribute.

This is completely optional; python does not attach any meaning or significance to it. However, it can be used by third-party applications to enhance their operations. IDE such as Visual Studio Code can provide type-checking and code hinting. It can be also be used for foreign-language bridges, predicate logic functions database query mappings etc.

```
def «function_name»(«list_of_argument(s)»):
    Body of function
```

```
>>> def hello(name: str) -> None:
...   print(f'Hello world, from {name}')
...
>>>
>>> hello('ilia')
Hello world, from ilia
```

If a function does not explicitly have a return statement, then a None type is returned.

```
>>> def foo( ):
...   a = 1 + 3
...
>>>
>>> print(foo())
None
```

The above example demonstrates “type hinting”, and it makes your code more user friendly. Although type hinting is optional, we will try to use it as much as possible in this course.

5.a.9 – Recursive Functions

A function that calls itself is a recursive function. A popular example of recursion is the factorial function which is defined by the following two statements:

$F_1 = 1$

$F_n = n * F_{n-1}$

```
def fact(n: int) -> int:      #annotations
    if n == 1:
        return 1
    else:
        return n * fact(n-1)

n = 5
print(f'The factorial of {n} is {fact(n)}')
```

Another example of a recursive function is the Fibonacci function which is defined by the following three statements:

$F_0 = 0$

$F_1 = 1$

$F_n = F_{n-2} + F_{n-1}$

> Challenge
>
> Can you implement the factorial or the Fibonacci functions without recursion?

```
def fib(n: int) -> int:
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fib(n-2) + fib(n-1)
n = 8
print(f'The {n}th fibonacci number is {fib(n)}')
```

Recursion provides elegant implementation for lots of classical problems in computer science such as searching and sorting.

5.a.10 – Inner Functions

A function that is declared within another function is an inner function or a nested function. The inner function has access to the variable of the enclosing function so it is often used as helper functions. It can also be used to provide encapsulation or hiding the inner function. Unless the inner function is extremely short it is a better practice to use private helper functions. When deciding to use inner function, you should consider code readability and maintainability. You might think of using lambda expression instead.

Another popular use of inner function is closures that is covered in the part of this section.

You should avoid using inner-function, because the problems it can create far outweighs the benefits that you might get using it.

```
def talk(phrase: str) -> None:
    def say(word: str) -> None:      #inner function
        print(word)

    words = phrase.split()
    for word in words:
        say(word)

>> talk('COMP216 is the best course')

COMP216
is
the
best
course
```

5.a.11 – Higher-order Functions

In Python functions are first-class objects. They can be stored as variables, passed as arguments and returned from other functions.

```
def shout(text: str) -> str:      #annotations
    return text.upper()

def whisper(text: str) -> str:
    return text.lower()

txt = 'Hi Narendra'
shout(txt)                        #HI NARENDRA
yell = shout                      #assigns function to a variable
yell(txt)                         #HI NARENDRA

whisper(txt)                      #hi narendra
```

```
def greet(func) -> str:           #function that takes a function argument
    return func('from a higher power')

print(greet(shout))               #FROM A HIGHER POWER

print(greet(whisper))             #from a higher power


'''
A function that returns a function.
'''

def make_entity(kind) -> (str):
    def client():                 #a nested function definition
        return 'I am a client'

    def server():
        return 'I am a server'

    if kind == 'client':
        return client
    else:
        return server

kind = 'client'
f = make_entity(kind)
print(f())                        #I am a client

kind = 'some_thing'
f = make_entity(kind)
print(f())                        #I am a server
```

5.a.12 – Closures

A closure is a function that remembers values from its enclosing scope even after that scope has finished executing.

A closure must satisfy the following:

- There should be nested function i.e. function inside a function.
- The inner function must refer to a non-local variable or the local variable of the outer function.
- The outer function must return the inner function.

```
'''
A function that returns a closure.
'''

def adder(x) -> (int):
  def add(y) -> int:
    return x + y

  return add                    #returns a closure of the captured x

incrementBy2 = adder(2)           #a closure that adds 2
incrementBy3 = adder(3)           #a closure that adds 3
incrementBy9 = adder(9)           #a closure that adds 9

print(incrementBy2(3))            #5
print(incrementBy3(7))            #10
print(incrementBy9(30))           #39
```

5.a.13 – Lambda Closures

A closure is a function object (a function that behaves like an object) that remembers values in enclosing scopes (variables etc.) even if they are not present in memory. An inner function is able to access the (non-local) variable of the outer function.

A closure must satisfy the following:

- There should be nested function i.e. function inside a function.
- The inner function must refer to a non-local variable or the local variable of the outer function.
- The outer function must return the inner function.

```
'''
A function that returns a closure.
'''
```

5.b – Documentation

Explains the purpose and function of the code.

```
#This function calculates the area of a rectangle
def calculate_area(length, width):
  return length * width
```

5.b.1 – Code Explanation

To provide clarity on complex logic or non-obvious code segments or workarounds.

```
#Using list comprehension to get the multiples of 3 less than 20
multiple = [x for x in range(20) if x % 3 == 0]
```

5.b.2 – Disabling Code

To temporary disable or comment out a section of code for testing or debugging.

```
#print('This line will not be processed by the interpreter')
```

5.b.3 – To-Do Notes

To mark section of code that need further attention such as pending tasks or later improvements.

```
#TODO: Implement error handling for invalid input
def divide(a, b):
  return a / b
```

5.b.4 – Code Maintenance

To help developer in the future understand the code history and rationale.

```
# This workaround is necessary due to a known bug in the library
if library_version < '1.0.0':
    # Use the old method
    result = old_method()
else:
    # Use the new method
    result = new_method()
```

5.b.5 – Code Review

To add notes during code review processes, helping reviewers understand specific decisions or changes.

```
# Reviewer note: This check is necessary to prevent division by zero
if denominator != 0:
    result = numerator / denominator
else:
    result = 0
```

5.b.6 – Debugging Information

To leave notes that might help in debugging, such as variable values or conditions.

```
# Debug: Checking the value of variable x
x = 10
# print(f"x is {x}")
```

5.b.7 – Best Practices for Writing Comments

Be Clear and Concise: Write comments that are easy to understand and to the point.

Update Regularly: Keep comments up-to-date as the code changes. Outdated comments can confuse readers and lead to bugs.

Avoid Redundant Comments: Don't comment on obvious code. Instead, focus on explaining complex logic or non-obvious decisions. Do not explain how but why.

Use Consistent Style: Follow a consistent style for comments to maintain readability.

Use Docstring for Documentation: Use docstrings for modules, classes and functions to describe their purpose, parameters and return values. (See the appendix for a more thorough coverage.)

By using comments effectively, you can make your code more maintainable, understandable, and easier to debug.

5.c – Built-in Functions

These are functions that are available in all instances of python without you explicitly coding it. They are very useful for both the novice and the expert programmer. Some of the more useful ones are:

- **`id(x)`**
  returns the id of the argument. In cpython (the normal implementation) it is normally the object's memory address.

- **`type(x)`**
  outputs the underlying type of the argument.

- **`del(x)`**
  removes the argument from the current environment.

- **`dir(x)`**
  outputs all the members (data as well as functional attributes) of the argument.

  If this function is invoked without any arguments, it will list all the variables in the current environment.

- **`help(x)`**
  outputs same the same as the above with additional explanation on each member.

- **`print(x)`**
  outputs the stringified version of the argument.

- **`isinstance(obj, type)`**
  returns a bool indicating if the first argument is of type of the second argument.

- **`hasattr(obj, attr)`**
  returns a bool indicating if the first argument has a member of type of the second argument. The second argument is of type string.

- **`getattr(obj, attr)`**
  returns the value of the attribute specified by the second argument. If the attribute does not exist on the object then an exception is raised.

- **`dir(__builtins__)`**
  returns a list of all the built-in function available in the default environment of the current python distribution.

Some other useful functions are: **`str()`**, **`int()`**, **`float()`**, **`abs()`**, **`round()`**, **`max()`**, **`min()`**, **`len()`**, **`pow()`**, **`ord()`**, **`strip()`**, **`upper()`**, and many, many more.

1.e – Scope

Scoping refers to the rules that determine where a particular variable is accessible and what value it holds. Understanding these rules is essential for writing correct and maintainable code. The python interpreter follows LEGB (Local -> Enclosing -> Global -> Builtin) when executing a program.

1.e.1 – Local Scope

This is the innermost scope, typically associated with a function or method.

```
def foo():
    a = 5
    print(f'a: {a}')

print(f'a: {a}')    #a in not accessible here
```

1.e.2 – Enclosing Scope

This deals with nested functions.

```
def outer():
    a = 5

    def inner():
        print(f'a: {a}') #a is in an enclosing scope

    inner()

outer()
```

1.e.3 – Global Scope

Global scope in the outermost scope of a program. To modify a global variable you must use the global keyword.

```
CURRENT_ID = 1_000

def generate_id ():
```

```
global CURRENT_ID      #the global keyword is necessary here

CURRENT_ID += 1
```

1.e.4 – Built-in Scope

This is the widest scope! All the special reserved keywords falls under this scope. These can be accessed directly without any imports.

Randomisation

The random functions come from the random module which needs to be imported. In this case, the start and end are both included:

start <= randint <= end

```
import random
# randint(start, end)
n = random.randint(2, 5)
#n can be 2, 3, 4 or 5.
```

range

Often you will want to generate a range of numbers. You can specify the start, end and step. Start is included, but end is excluded:

start >= range < end

```
# range(start, end, step)
for i in range(6, 0, -2):
  print(i)          # displays 6, 4, 2 but not 0
```

round

This does a mathematical round. So 3.1 becomes 3, 4.5 becomes 5 and 5.8 becomes 6.

```
round(4.6)          #result 5
```

abs

This function work with numeric types, it returns the absolute value of its argument. Basically, removing any negative signs.

```
abs(-4.6)           #result 4.6
```